HALCYON HACKATHON · DELIVERY DIAGNOSTIC

Fix the five causes, not just the carrier

What really makes 29% of Halcyon deliveries late, how to fix it, and how we will build and prove it

71.0%

on-time delivery today

92.0%

projected after fixing all five causes

Evidence: 48,340 deliveries · 280 clients · 5 warehouses · 6 carriers · 1 Apr – 30 Sep 2026

Executive summary

71.0%

on-time today, flat for 6 months

5 causes

explain 100% of the 14,019 late deliveries

+14 pp

more on-time than the routing vendor alone (92% vs 78%)

$1.35M

redelivery + penalty cost in 6 months, 1.8× freight spend

1

Carriers are a quarter of the problem, and only on specific lanes

Carriers D, E and F fail on five region lanes and D collapses on Fridays. A, B and C run about 5% late everywhere.

2

The other three quarters start before a truck is booked

Incomplete addresses from email keying, a warehouse fallback that ignores distance, and same-day promises accepted after the 08:00 cutoff.

3

Recommendation: run a 16-week diagnostic and fix programme first

Build a control tower that flags risk at order intake and recommends the matching fix. Decide on the routing vendor at the end, with evidence.

1

What the data says

Profile, quality and the shape of the problem

Eight files, one delivery table at the centre

File	Rows	What it holds	Key use
deliveries.csv	48,340	Order → pick list → ship → deliver, per delivery	Stage timing, cause flags, model features
clients.csv	280	Tier, revenue, SLA clause, account status	Revenue at risk, prioritisation
carriers.csv	6	API vs portal vs batch feed, base rate, preferred supplier	Visibility gaps, lane cost
warehouses.csv	5	08:00 pick-list cutoff, 14:00–18:00 collection	Promise feasibility
warehouse_region_distance	5	Miles from each warehouse to each region	Fallback warehouse ranking
sla_penalties.csv	4,847	Penalty per late delivery (flat $180)	Cost of failure
complaints.csv	34	Last 90 days: category, text, escalation	Voice of the customer
customer_success_time_study	18	One week of CS hours by activity	Hidden people cost

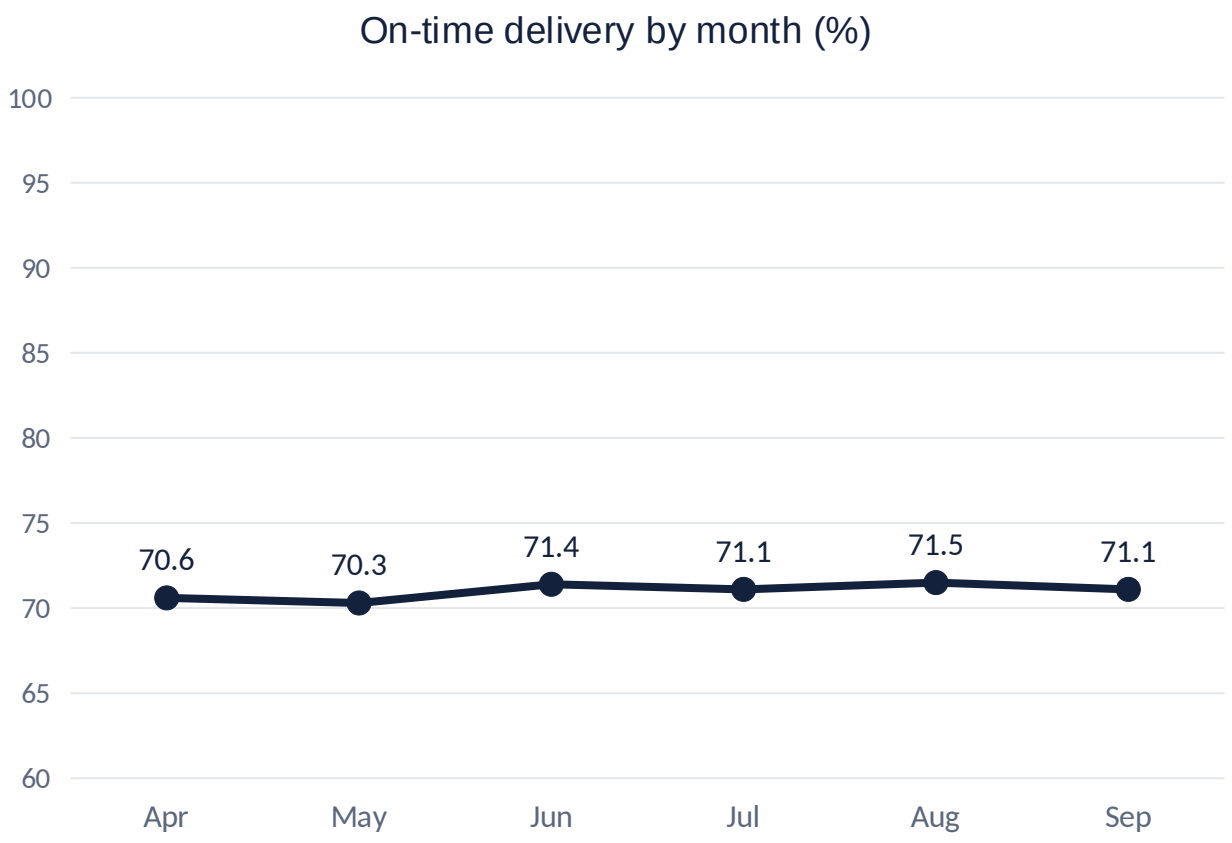
48,340
deliveries

$105.2M
annual client revenue

3 channels
API 36% · EDI 32% · Email 32%

6 months
1 Apr – 30 Sep 2026

Failure is structural, not seasonal or regional



Late rate barely moves across any single dimension

- Region: 28.9% – 29.1% for all four
- Fulfilment warehouse: 28.2% – 29.6%
- Order channel: 28.5% – 29.3%
- Account tier: Tier 1 29.1% = Tier 3 29.1%
- Month: 70.3% – 71.5% on-time

So no single team or site is "the problem". The causes sit inside each order's journey, which is why we diagnose shipment by shipment.

Note: the stakeholder call quoted ~75% on-time; the data shows 71.0% (34,321 of 48,340). We use the data and flag the gap as an open question.

Raw records need care before any KPI is trusted

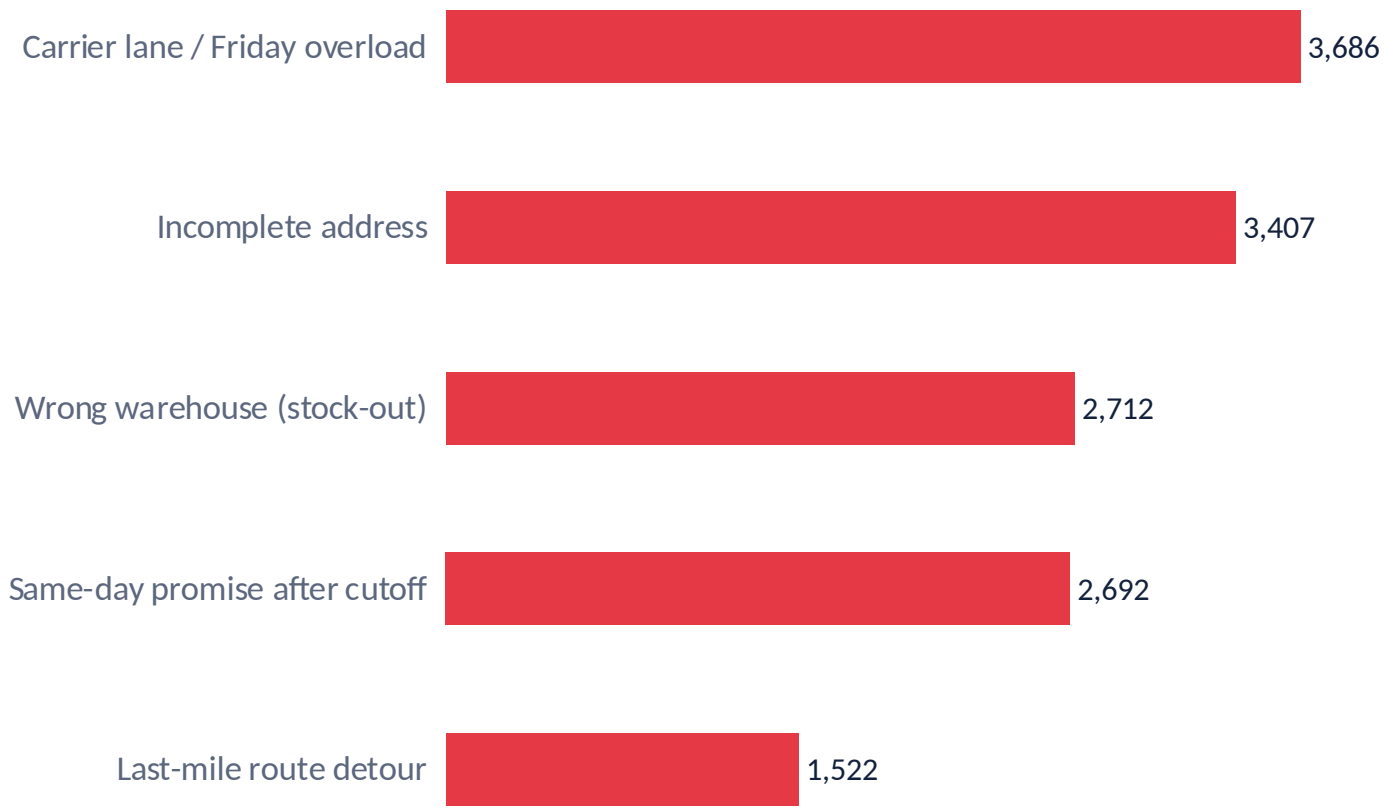
Issue found	Evidence	Impact on the solution
Email timestamp = keying time	Email orders only 06:00–16:59; 78% land after cutoff vs 55% for API/EDI	True receipt time is lost. Capture it at intake
450 impossible deliveries	Same-day orders delivered before they were picked or shipped, all marked on time	Quarantine; they inflate on-time delivery
Collection window contradicted	59% of shipments collected before 14:00 vs a 14:00–18:00 window	Confirm what shipped_ts means
“same_day” label	Promised date is always receipt + 1 day (next_day +2, standard +4)	Rename or redefine promise classes
Redelivery cost = the label	$34 on every late row, $0 on every on-time row	Exclude from model features (leakage)
Penalties incomplete	Flat $180; only 70.8% of eligible late deliveries penalised (2,002 missing)	~$360k waived or unbilled: ask finance
No pick/pack timestamps	Ship date always equals pick-list date	Packing delay cannot be measured; add milestones
Smaller gaps	1,339 blank actual_route_km · 21 duplicate order_ids · CS hours ~2h short	Mark “unknown”, never assume

2

Where shipments become late

Five root causes, each sized, each with its own evidence

Five causes explain every late delivery



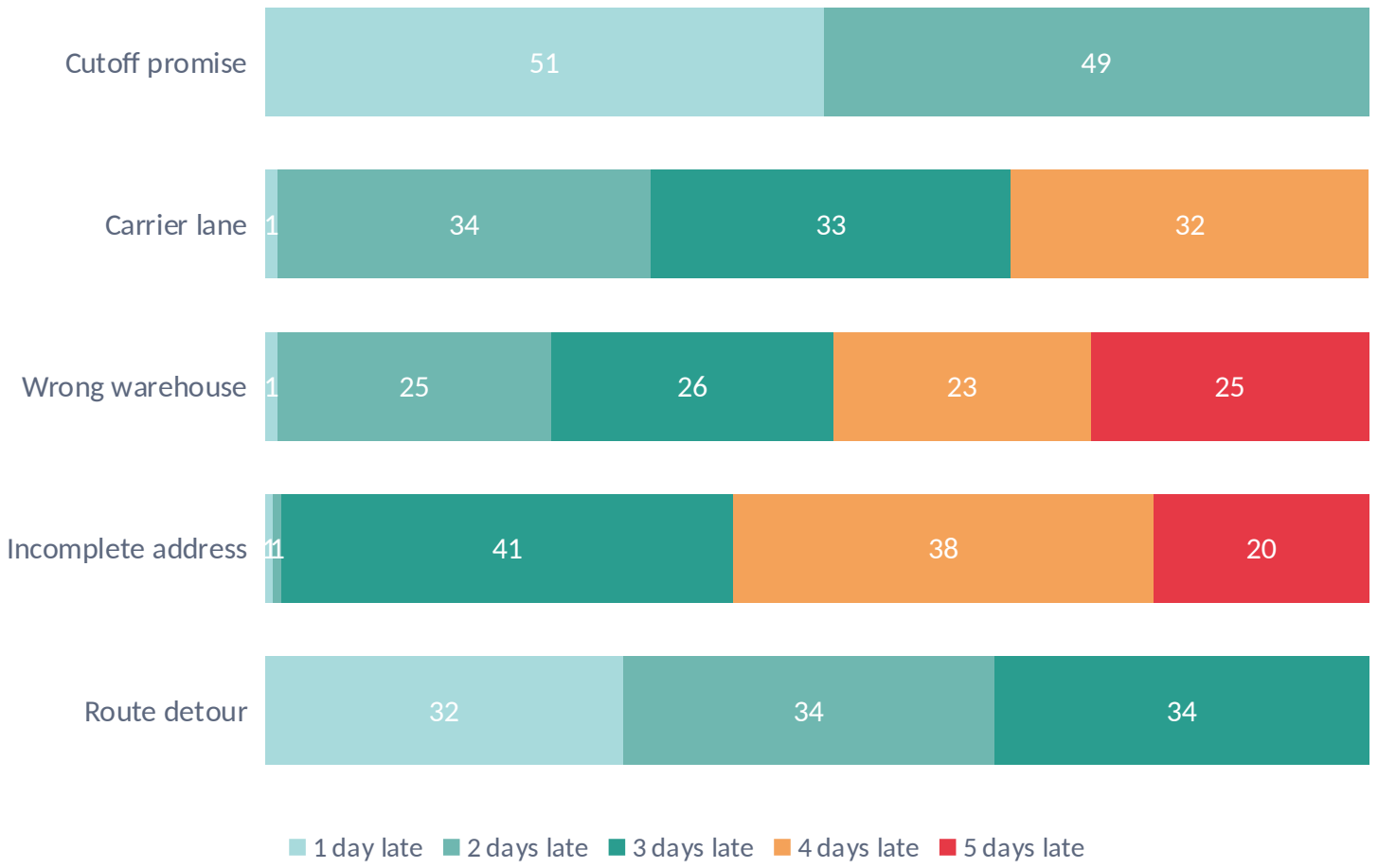
Points of on-time lost

Carrier lane	**7.6 pp**
Address	**7.0 pp**
Warehouse	**5.6 pp**
Cutoff promise	**5.6 pp**
Route detour	**3.2 pp**
Total	**29.0 pp**

Method: Shapley attribution over the five cause flags, so shipments with two causes share the blame fairly.

With all five flags off, the late rate is 0% in this dataset: the five causes are the complete explanation. Carrier choice is the largest single cause, but only 26% of the total.

Each cause leaves a fingerprint in days late



Share of late deliveries by days late (%), shipments with a single cause only.

Why this matters

- Cutoff misses are always 1–2 days late: the order simply waited a day
- Carrier failures run 2–4 days, never 5
- Address failures cluster at 3–5 days: a failed attempt, then redelivery
- Distant warehouses spread evenly from 2 to 5 days

The diagnosis engine uses this pattern as evidence for each cause it names.

Carriers fail on specific lanes, not everywhere

	Midwest	Northeast	Southeast	West
Carrier A	4%	4%	6%	6%
Carrier B	5%	5%	5%	5%
Carrier C	5%	4%	4%	4%
Carrier D	46%	22%	44%	22%
Carrier E	5%	3%	5%	46%
Carrier F *	4%	43%	46%	5%

Late rate on clean shipments (complete address, nearest warehouse, feasible promise). * F = preferred supplier

5 bad lanes

D → Midwest & Southeast · E → West · F → Northeast & Southeast, each 43–46% late

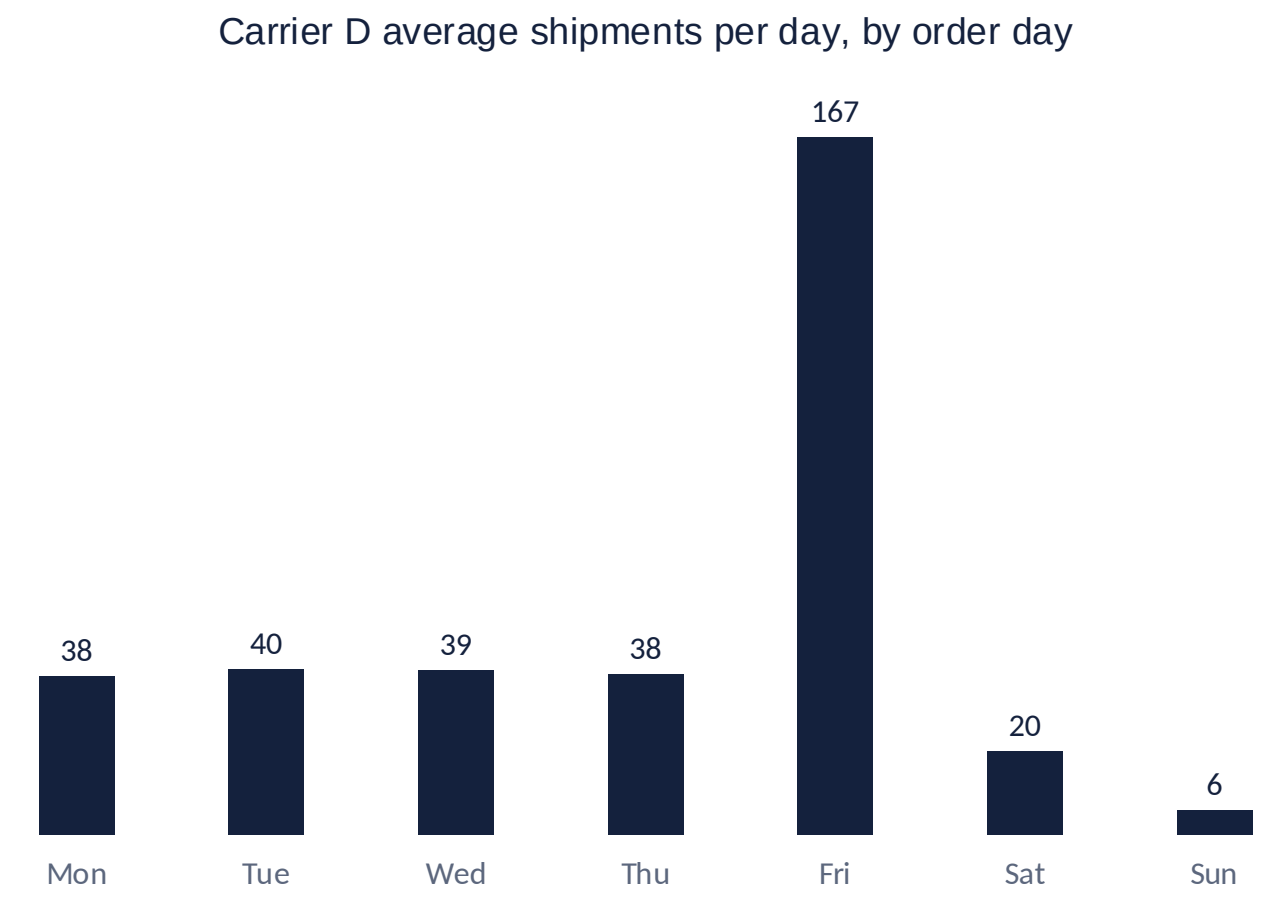
F is the preferred supplier

Cheapest base rate ($10.50), but 46% late into the Southeast and 5% into the Midwest.

Fix: a lane rule table, not a new platform

Moving 10,030 at-risk shipments to A/B/C adds ~$30k freight and avoids ~$355k of redelivery and penalties.

Fridays quadruple carrier D’s load, and it breaks



+37%

Friday order volume vs other weekdays (430 vs 313 a day)

12% → 39%

D’s share of all shipments on Fridays

25% → 44%

D’s late rate on its normally healthy lanes on Fridays

Fix: a Friday capacity cap on D with overflow to A/B/C, plus a Friday pre-book with carriers. Scheduling, not route optimisation.

Stock-outs ship cross-country, not next door



Same pattern for W2: 0 orders go to W1. Other warehouses spread fallbacks evenly regardless of distance.

Fix: rank fallback warehouses by distance (sibling first) and rebalance stock weekly.

12.5%
nearest-warehouse stock-out rate, flat everywhere

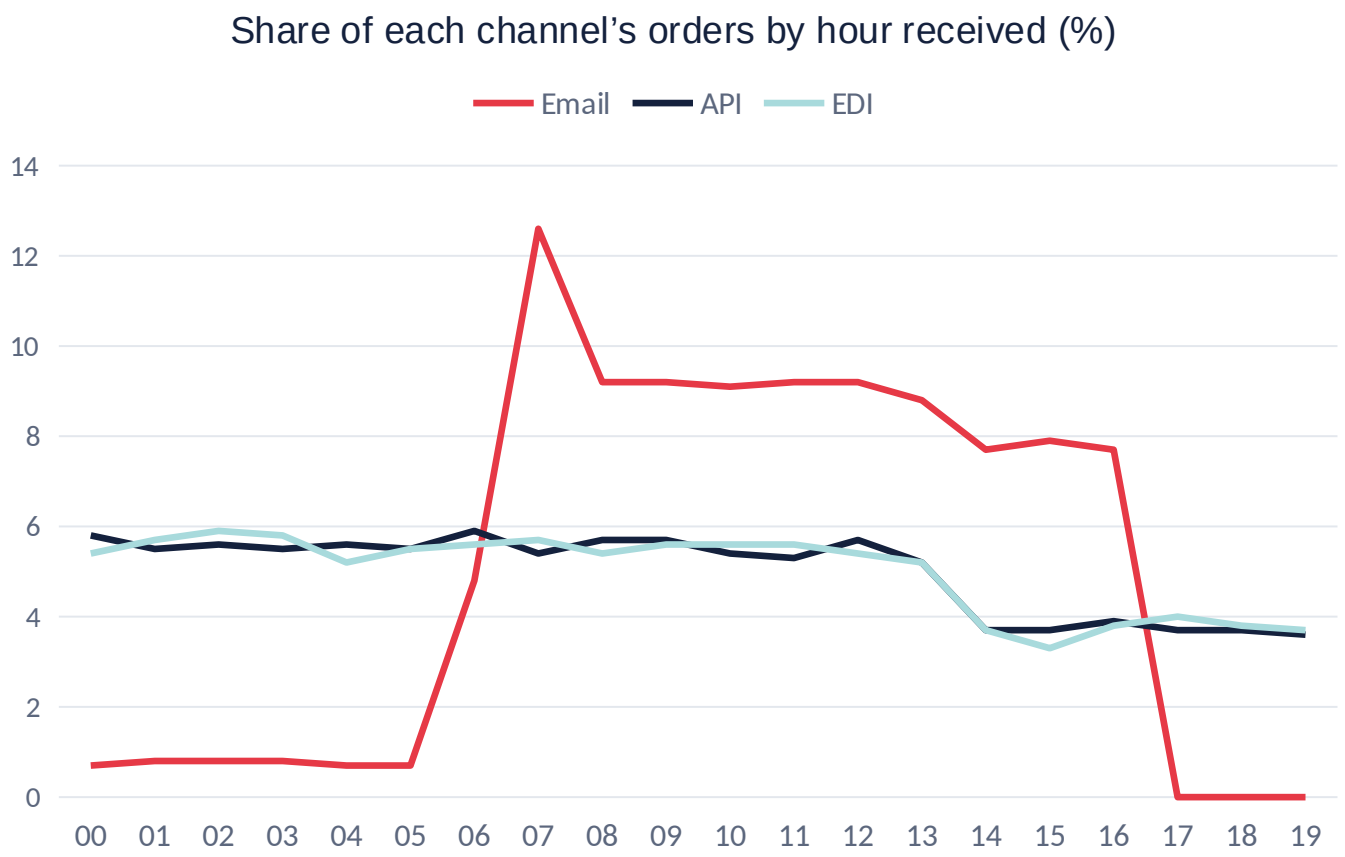
1,238 vs 127 mi
average distance, wrong vs right warehouse

$23.58 vs $13.93
freight per shipment: distance costs money too

50% vs 26%
late rate, wrong vs right warehouse

691
shipped far away although the nearest site had stock

Email orders arrive late, with broken addresses



61% vs 26%

late rate of same-day orders received after vs before the 08:00 cutoff

27.8% vs 5.3%

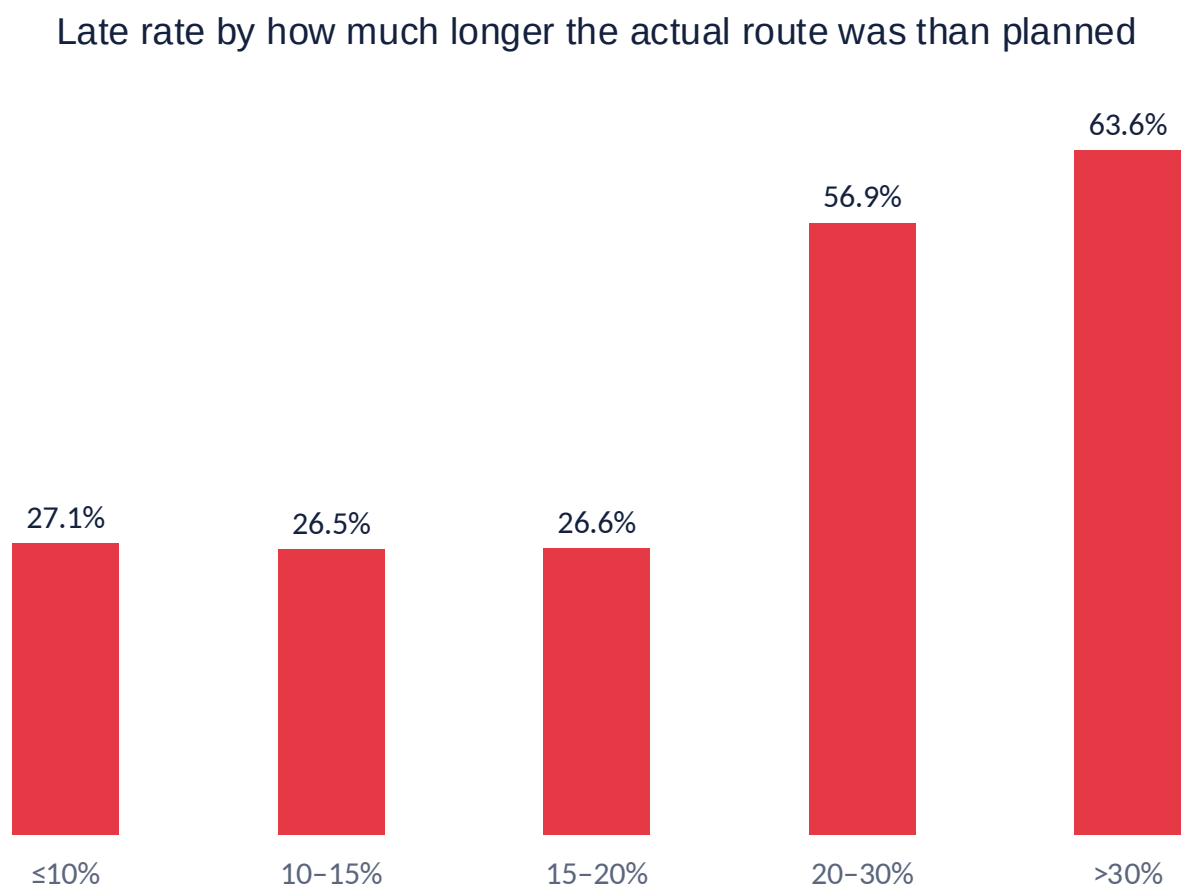
incomplete addresses: email vs API/EDI, equal across all 14 email layouts

4.4 min

keying time per email order (44 h a week)

Email “receipt” timestamps only appear in office hours, so overnight emails are probably keyed after the cutoff. Fix: AI extraction plus address validation at intake, and cutoff-aware promises.

Route detours over 20% double the late rate



A last-mile execution problem

- 2,880 deliveries ran more than 20% over the planned route
- Detour rate is 10–11% for every carrier, region and weekday
- On otherwise clean shipments, no delivery is late below a 20% overrun
- This is the one cause where in-route optimisation genuinely helps, worth 3.2 pp

Fix: live overrun alerts and dynamic re-sequencing for drivers.

Late deliveries cost more than the freight

$476.6k

redelivery cost, 14,019 × $34

$872.5k

SLA penalties, 4,847 × $180

$738k

total carrier freight spend, same 6 months

$14.2M

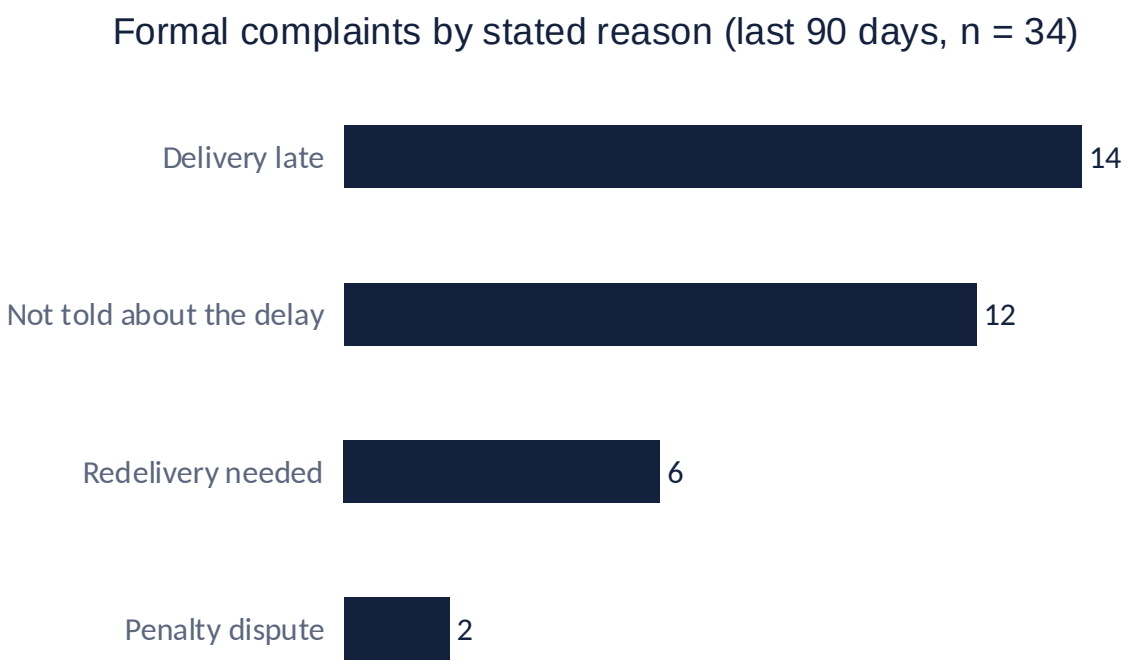
annual revenue from 9 clients threatening to leave



Failure costs ($1.35M) are 1.8× the freight bill. Saving a few dollars per shipment on carriers cannot pay for this.

64% of customer success hours go to complaints. Every percentage point of on-time gained frees capacity for proactive account work.

At-risk clients are uninformed, not later



Carriers D, E and F send no live events (portal or batch file only), so Halcyon learns of delays after the client does.

7 of 18

Tier 1 accounts threatening to leave

29.3% vs 28.9%

late rate, at-risk vs active clients: no difference

48% vs 28%

share of orders by email, at-risk vs active

24 of 34

complaints come from at-risk accounts; 26 escalated

Service is tier-blind today: Tier 1 runs 29.1% late, the same as Tier 3. Prioritisation and proactive alerts protect revenue fastest.

3

The solution

A delivery control tower that diagnoses, predicts and recommends

Baseline workflow kept, then grounded in data

Kept from the baseline

- Diagnose before buying the routing tool
- Match each fix to its cause
- Green / amber / red / late risk status
- Human in control: accept, override, give a reason
- Learn from every outcome

Changed or added from the data

- Dropped pick/pack and missed-pickup causes: not measurable here
- Added address quality, distance-blind fallback and Friday overload
- Sized every cause in percentage points
- Added client alerts for carriers without live events
- Added tier-aware prioritisation and revenue at risk
- Added a what-if simulator to prove results before go-live

The guiding question: where did the shipment become late, why, and what action can still prevent it?

End-to-end workflow: a fix wherever delay starts

1	2	3	4	5	6	7
Capture	**Validate**	**Allocate**	**Select carrier**	**Score risk**	**Act & alert**	**Learn**
Log true receipt time; AI-extract email orders	Address check; promise vs cutoff and transit	Nearest stocked warehouse, sibling first	Lane reliability table, Friday cap on D	Model scores every order at intake	Red queue, accept/override, client notice	Outcome, cause, override reason → retrain
Email keying	Address · Cutoff	Wrong warehouse	Carrier lane	All causes	Visibility	Continuous

Milestones recorded for every shipment: order received → keyed → address valid → warehouse allocated → pick list → shipped → carrier events → delivered or exception. Today's data has only four of these.

Every cause gets its own fix, owner and measure

Cause	Detect at	Recommended action	Owner	KPI to prove it
Incomplete address (7.0 pp)	Intake	Validate and geocode; hold and fix before 08:00 pick list	Customer success	% incomplete at pick list → <2%
Carrier lane / Friday (7.6 pp)	Carrier booking	Lane rule table; cap D on Fridays; overflow to A/B/C	Transport planning	Bad-lane share → 0; late rate by lane
Wrong warehouse (5.6 pp)	Allocation	Distance-ranked fallback; weekly stock rebalancing	Inventory planning	Avg miles; stock-out rate <8%
Cutoff promise (5.6 pp)	Promise	Cutoff-aware promise or midday pick wave	Sales ops / warehouse	Same-day after-cutoff late rate
Route detour (3.2 pp)	In transit	Overrun alerts, re-sequencing, proactive notice	Carrier management	Share of routes >20% over plan
Not told about delay	Any red	Auto-notify Tier 1 and at-risk clients first	Account owners	“Not told” complaints → 0

pp = percentage points of on-time delivery currently lost to that cause (Shapley attribution).

Risk is predictable at order intake

AUC 0.83

trained Apr–Jul, tested Aug–Sep, pre-dispatch features only

Risk threshold	Shipments flagged	Late deliveries caught	Precision
≥ 0.15 (amber)	50%	92%	53%
≥ 0.30	46%	88%	55%
≥ 0.50 (red)	28%	58%	60%

GREEN

Expected on time

AMBER

Watch; act if capacity allows

RED

Act before the 08:00 pick list

LATE

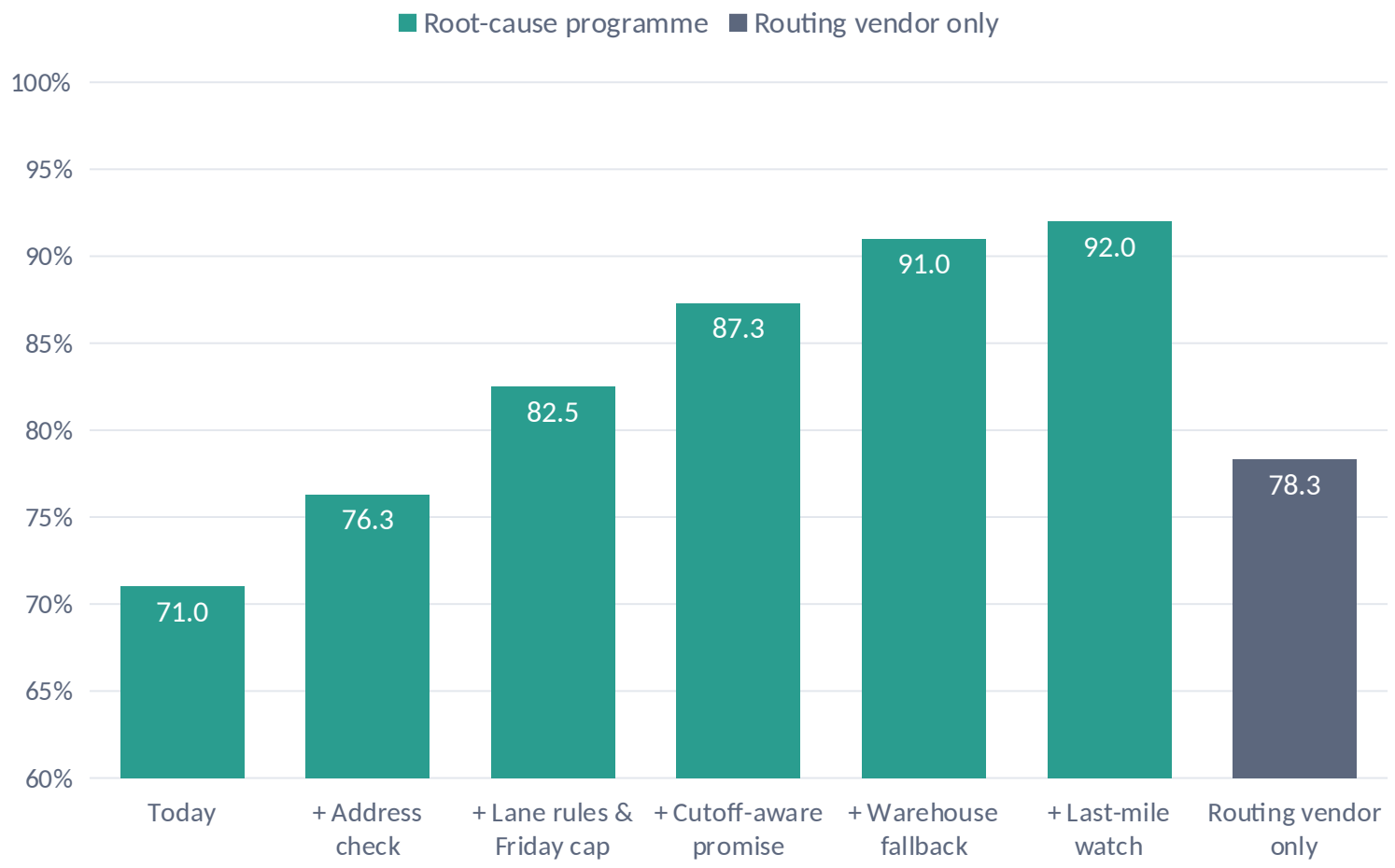
Promise missed: notify, recover

Features used

Address status · nearest vs fulfilment warehouse · distance · commitment type · receipt hour and weekday · channel · carrier × region · stops on route.
Excluded: redelivery cost (leaks the answer) and anything recorded after dispatch.

Model: gradient-boosted trees (LightGBM / scikit-learn) with SHAP explanations, so every score shows the reasons behind it.

Proof: replaying six months under each fix



Adoption assumed

- Address issues fixed: 85%
- Bad-lane shipments moved: 90%
- Cutoff cases recovered: 85%
- Wrong-warehouse avoided: 60%
- Detours prevented: 30%

Vendor case assumes it fixes every carrier issue and 30% of detours: a generous upper bound.

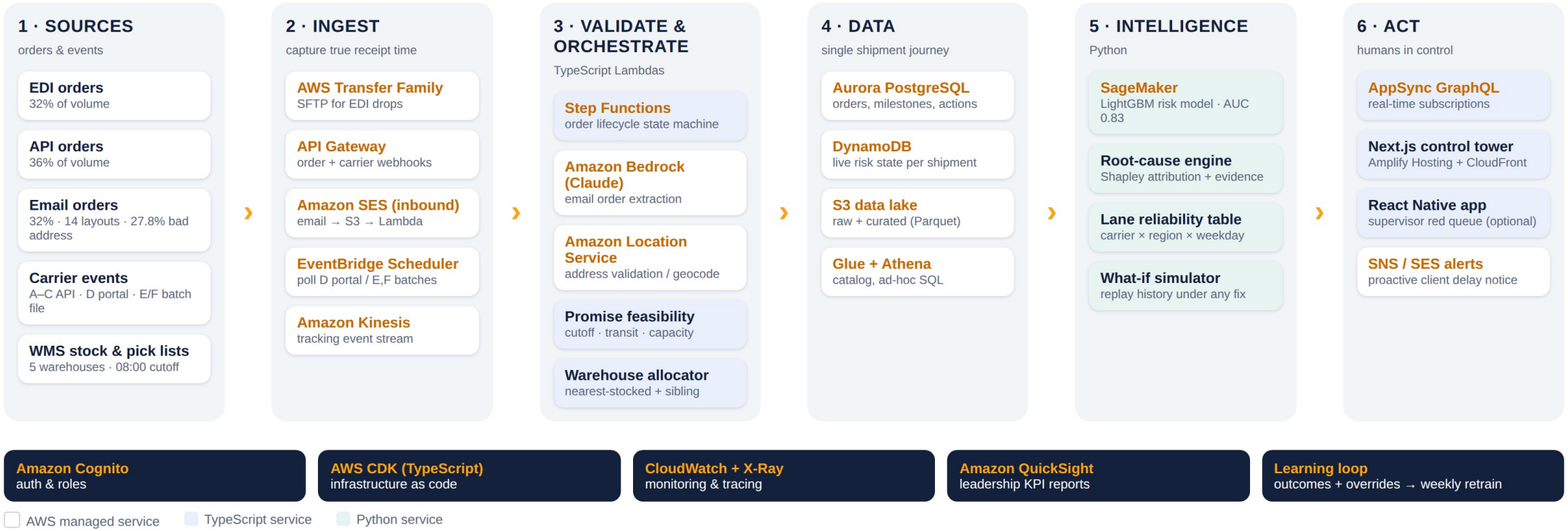
Counterfactual replay of 48,340 historical deliveries using late rates of matching shipments without the fixed cause. Next step: a synthetic week of new orders in the MVP.

4

Build plan and technology

AWS · TypeScript · Python · optional React Native app

Target architecture on AWS

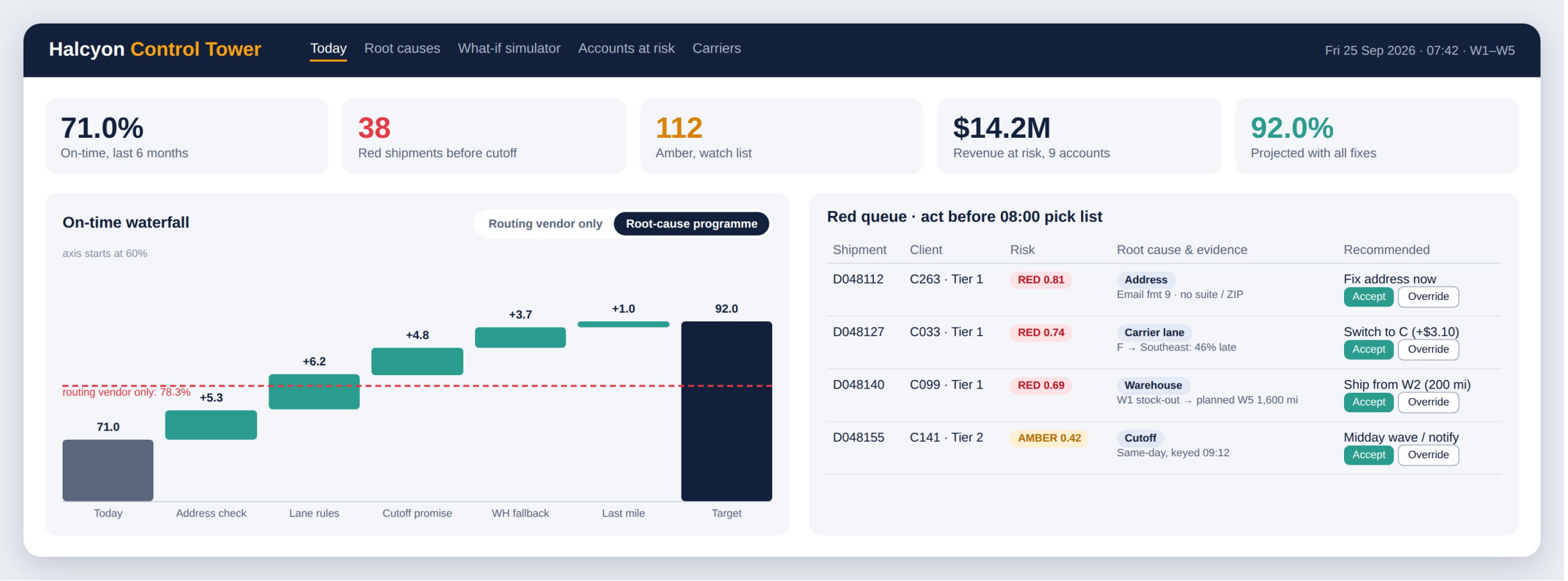


Event-driven and serverless first: each order is a Step Functions execution, so every milestone is timestamped by design. Python owns analytics and ML; TypeScript owns services, UI, mobile and infrastructure.

Recommended technology stack

Layer	Choice	Why
Infrastructure as code	AWS CDK in TypeScript	One language for infra and services; repeatable environments
Ingestion	API Gateway · Transfer Family (SFTP/EDI) · SES inbound · EventBridge	Covers all three order channels and the carrier portal and batch feeds
Orchestration & services	Step Functions · Lambda (Node.js/TypeScript) · ECS Fargate for heavier APIs	Pay-per-use; state machine mirrors the order lifecycle
AI document & address	Amazon Bedrock (Claude) · Amazon Location Service	Extract 14 email layouts; validate and geocode addresses
Data	Aurora PostgreSQL · DynamoDB · S3 + Glue + Athena	Transactions, live risk state, analytics lake
ML & analytics (Python)	SageMaker · pandas · LightGBM · SHAP · scikit-learn	Risk model, cause attribution, what-if simulator
API & real-time	AppSync (GraphQL subscriptions)	Live red queue on web and mobile
Web front end	Next.js + React (TypeScript) on Amplify Hosting / CloudFront	Control tower, simulator, account-risk views
Mobile (optional)	React Native (Expo) + SNS push	Shares TypeScript types with the web app
Security & ops	Cognito · IAM · KMS · CloudWatch · X-Ray · QuickSight	Roles per warehouse; monitoring; leadership reports

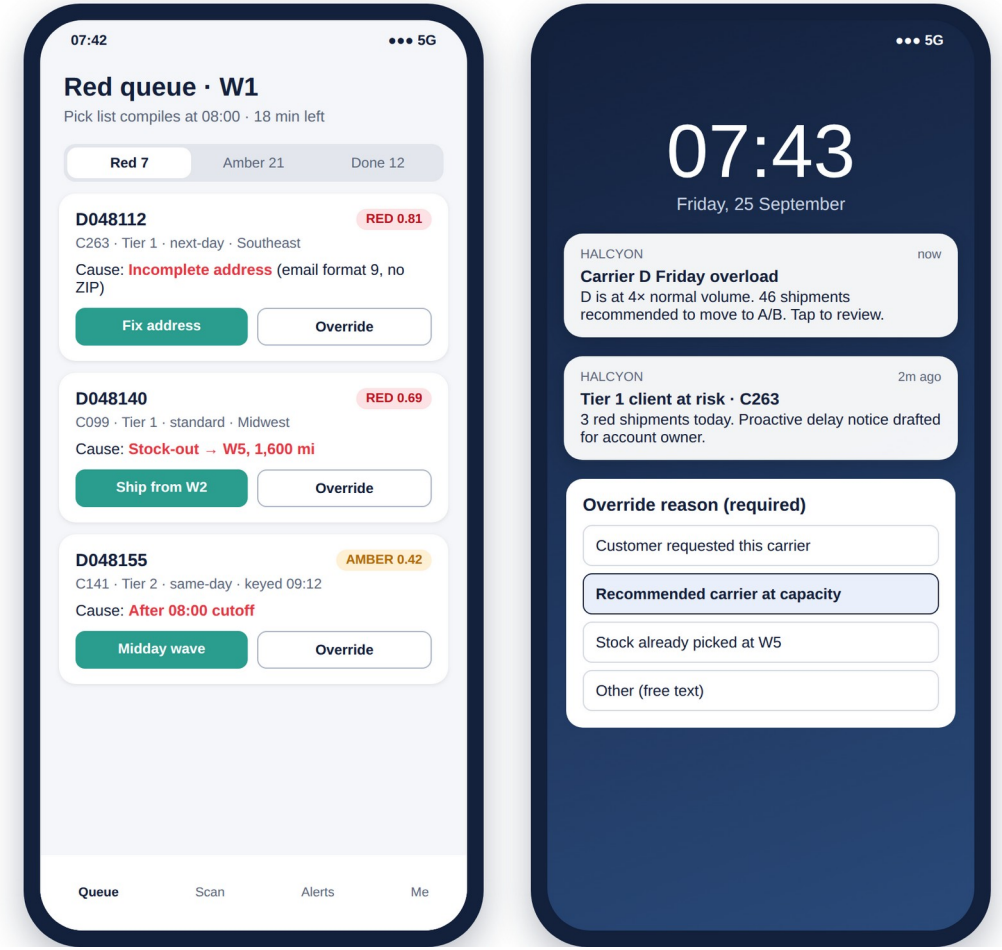
What leadership and warehouses will see



The “wow” moment: flip the toggle from routing vendor only to the root-cause programme and watch the forecast move from 78% to 92% on Halcyon’s own data. Each red shipment shows its cause, evidence and one-click action.

Illustrative mockup; shipment IDs in the queue are examples.

Optional mobile app: red queue in the pocket



Feasible, and best as phase 3

- Built in React Native (Expo), reusing the web app's TypeScript types and GraphQL API
- Supervisors clear red shipments before the 08:00 pick list from the floor
- Override needs a reason, which feeds the learning loop
- Push alerts for Friday overload and Tier 1 risk via Amazon SNS
- Later: drivers capture address fixes and delivery exceptions

Effort: about 3–4 weeks for one developer once the API exists. Not needed for the hackathon demo; show the mockup instead.

Hackathon MVP: built and demoed in 48 hours

0–8 h	**Data pipeline (Python)**	Clean the CSVs, quarantine anomalies, build the five cause flags and the data quality report
8–16 h	**Diagnosis & model (Python)**	Shapley attribution, days-late fingerprints, LightGBM risk model with out-of-time test
16–24 h	**Rules & simulator**	Address hold, cutoff promise, sibling fallback, lane rules; replay history plus a synthetic week of new orders
24–36 h	**API & UI (TypeScript)**	Lambda + API Gateway serving JSON from S3; Next.js control tower with waterfall, red queue and account risk
36–44 h	**Deploy on AWS**	CDK stack: S3, Lambda, API Gateway, Amplify Hosting, Cognito
44–48 h	**Rehearse**	Demo script: problem → evidence → fix → proof, with the vendor toggle as the finale

Out of scope for the demo: live carrier integrations, Bedrock email extraction (mocked with the dataset), mobile app.

Roadmap: about 92% in 16 weeks, then decide

Hackathon

48 h

Diagnostic and demo

Proof on historical data

—

Phase 1

Weeks 1–4

Quick wins, rules only

Address check at intake · lane rules · Friday cap on D · cutoff-aware promises

~82%

Phase 2

Weeks 5–10

Predict and alert

Risk model live · red queue · sibling fallback · client alerts · D/E/F event polling

~88%

Phase 3

Weeks 11–16

Automate and learn

Bedrock email extraction · stock rebalancing · mobile app · weekly retrain

~92%

Decision gate at week 16: if carrier and route issues still cost more than 3 pp, run the routing vendor as a pilot on the remaining lanes.

How we will prove each fix works

Measurement design

- Baseline frozen at 71.0% (Apr–Sep 2026)
- Roll out warehouse by warehouse; compare against not-yet-live sites
- Each fix has its own KPI, so gains are not double-counted
- Weekly cause waterfall: which cause shrank and by how much
- Synthetic order weeks to test rules before go-live

On-time delivery	**71.0% → 92%**
Redelivery + penalty cost (6 months)	**$1.35M → <$0.45M**
Incomplete addresses at pick list	**12.4% → <2%**
“Not told” complaints	**12 → 0 per quarter**
CS hours on complaints	**64% → <30%**
Tier 1 accounts at risk	**7 → 0**

Targets are projections from the replay; they will be re-baselined once the open data questions are answered.

Assumptions we made, and questions for Halcyon

Assumptions

- Data is representative of normal operations
- Email timestamps are keying time, not receipt
- Warehouses can run a midday pick wave
- Carriers A, B and C have spare lane capacity
- Missing penalties were waived, not lost
- Weekend picking and shipping is normal

Questions

- Is the quoted 75% on a different basis from the 71% in the data?
- How were 450 same-day orders delivered before shipping?
- What does shipped_ts mean against the 14:00–18:00 window?
- Why are 30% of eligible late deliveries not penalised?
- What does carrier F’s preferred-supplier agreement commit us to?
- Can we get pick, pack and dock timestamps from the WMS?

Fix the five first. Then decide on routing.

Approve a 16-week programme and a 48-hour proof, and keep the routing contract unsigned until the evidence is in.

1. Approve the control tower MVP on AWS
2. Pilot rules-only quick wins at W1 and W2
3. Answer the six open data questions
4. Re-decide on routing at week 16